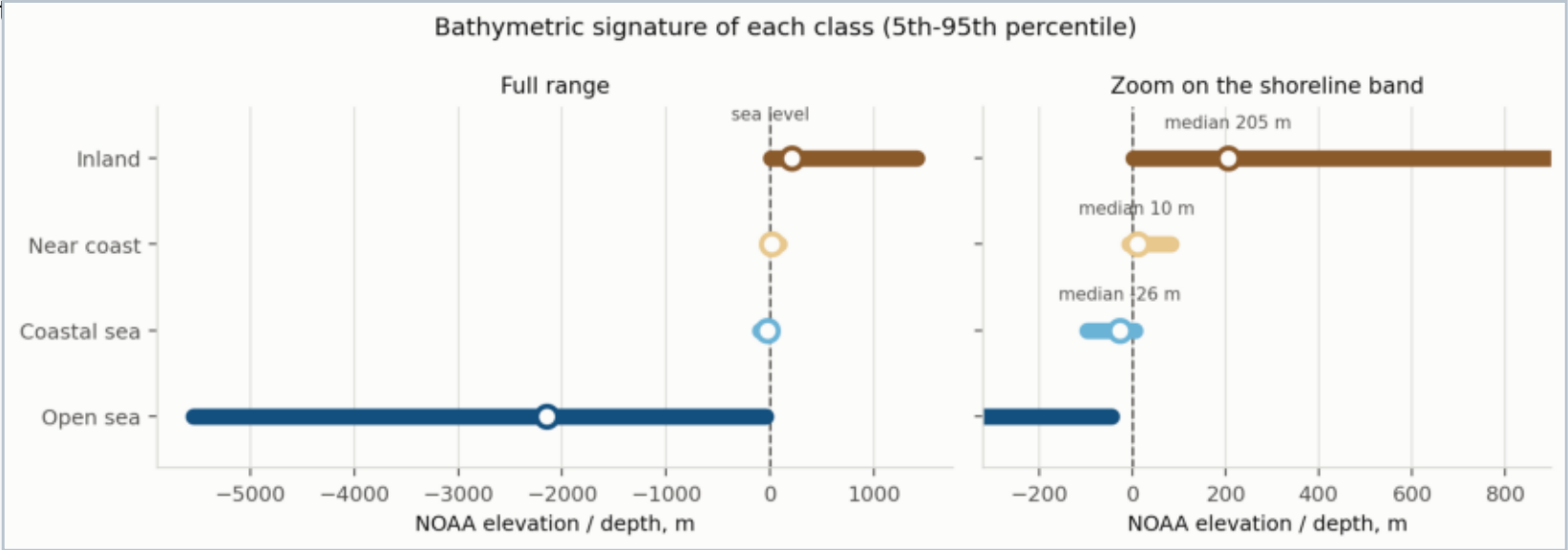


1. Purpose of the work

- Build an application that plans an optimal and safe maritime route between two points.
- The safety of every point of the route is decided by a neural network that classifies the geographic zone the point belongs to.
- The route is then produced by graph search on a cost map derived from those classes.
- The result is compared with the naive short



Why the task is non-trivial: the two shoreline classes occupy almost the same depth band

2. Input data

Labelled AIS corpus (merged.geojson)

- 147,763 geo-referenced points, 25 recording sessions
- Attributes per point: latitude, longitude, elevation, NOAA depth/height, 9 OSM object counters, timestamp
- Coverage: lat -40.1...67.1°, lon -87.9...168.3°

Run-time input

- Departure / destination coordinates, or an uploaded AIS track (GeoJSON / CSV / JSON)

3. Technology stack

- Python 3.11 · PyTorch 2.x (neural network)
- scikit-learn (scaling, spatial index, metrics)
- FastAPI + WebSocket · Leaflet (live map UI)
- SQLite (storage) · ReportLab (PDF export)

4. Geographic zones to be recognised

OPEN_SEA

Open sea - deep water far from any shoreline
traversal weight: 1.0

COASTAL_SEA

Coastal sea - navigable water close to the shore
traversal weight: 1.8

NEAR_COAST

Near coast - land within 5 km of the shoreline
traversal weight: impassable

COASTLINE

Inland - land remote from the sea

The first two classes are navigable, the last two are land and are assigned an infinite traversal cost, so the planner can never route a vessel through them. The ordering of the four classes along the depth axis is what the diverging colour ramp encodes.

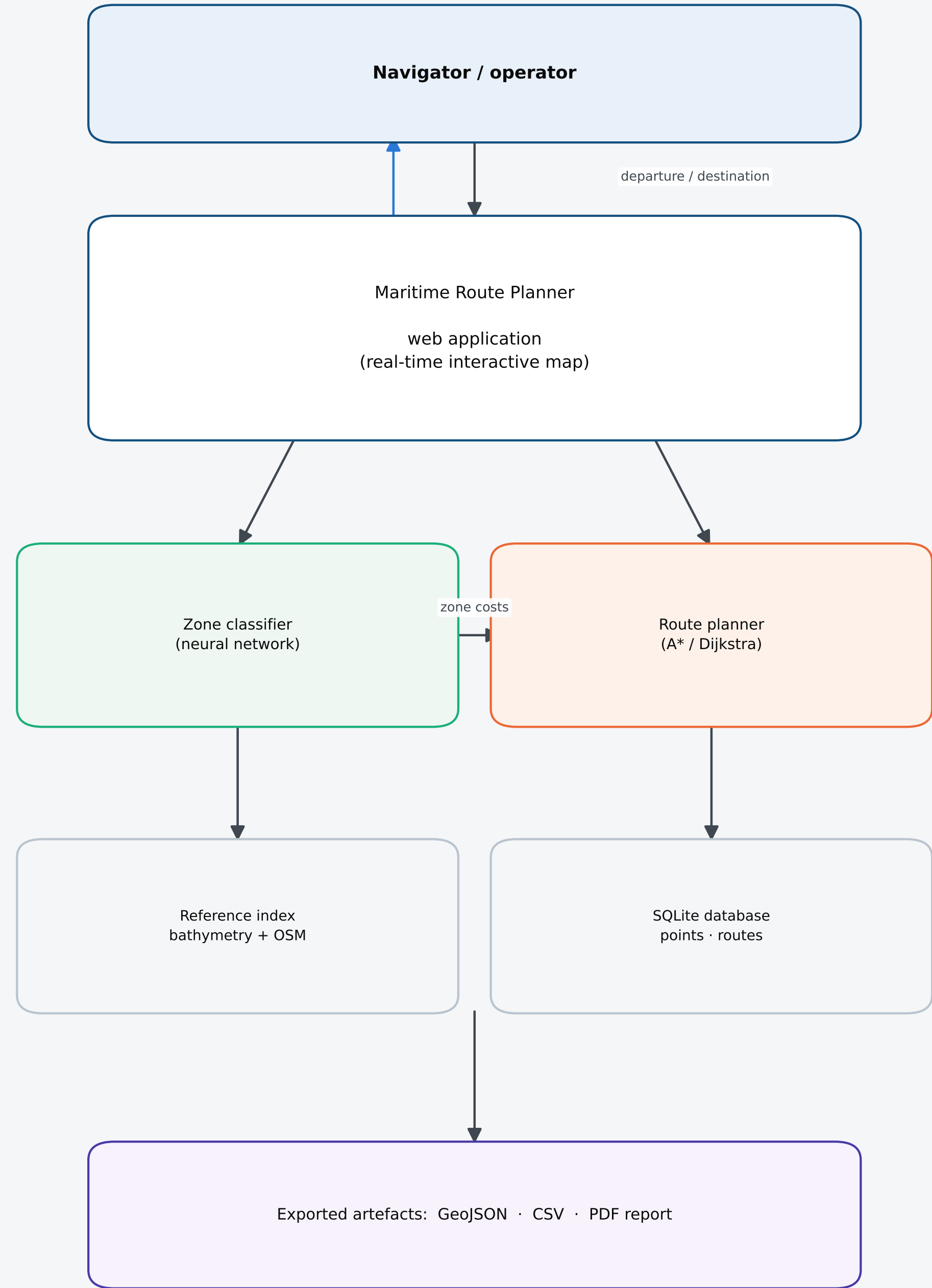
5. Functional requirements

- Load AIS data or route-point coordinates
- Classify geographic zones with a pre-trained neural network
- Build a traversal cost map from the classes
- Search the optimal route with A* or Dijkstra
- Visualise the route on an interactive map
- Compare with the shortest-distance route
- Export results to GeoJSON, CSV and PDF
- Persist points, zones, routes and their parameters in a database

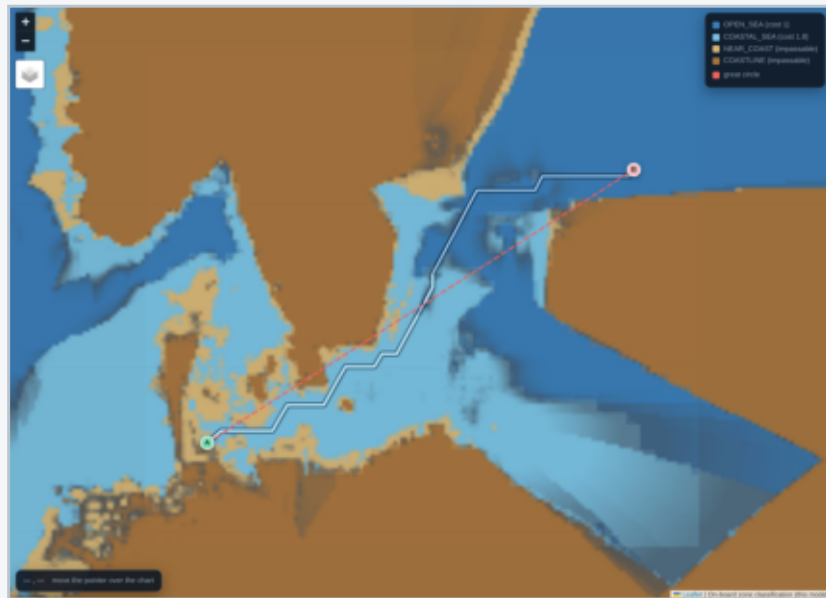
6. Quality targets and results

Criterion	Target	Achieved
Classification accuracy	≥ 90 %	96.62 %
Macro F1	≥ 0.90	0.9660
Route planning time	≤ 5 s	≤ 1.4 s
Routes free of land	100 %	100 % (6/6)

7. Context of use



The operator never sees raw zone codes: the application converts them into a traversal cost, searches the cheapest navigable path, and reports how much distance that safety costs compared with the straight great-circle line.



Planned route (blue) vs great circle (red dashed), Kiel → Tallinn